

Observation Task 4: Student Result and Performance Analyzer using Java (CO1)

Observation Task: 04

Date:

02/10/2026

■ Objective / Aim

To design and implement a **student performance analysis system** in Java using **Object-Oriented Programming principles** to calculate marks, percentage, grade, highest and lowest marks, and overall academic performance.

■ Problem Statement

Develop a Java application for a college to analyze the academic performance of a student. The system should accept the student's details and marks. It should perform the following operations:

- Store subject marks using an array.
- Calculate total and average marks.
- Find the highest and lowest marks.
- Calculate percentage.
- Assign a grade based on percentage.
- Determine whether the student has passed or failed.
- Display the student's name in a formatted manner using String operations.
- Round the percentage to two decimal places using the Math package.
- Display a simple performance remark based on the student's percentage.
- The program should use a class to represent the student and demonstrate basic object-oriented programming.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class named **Student**.
2. Declare the following data members:

- Roll number
 - Student name
 - Array of marks
3. Create a constructor to initialize the student details.
 4. Create methods to:
 - Calculate total marks.
 - Calculate average marks.
 - Find highest marks.
 - Find lowest marks.
 - Calculate percentage.
 - Determine grade.
 - Display student details.
 5. Store marks of five subjects in an integer array.
 6. Use a loop to calculate the total marks.
 7. Use Math.max() and Math.min() or suitable logic to determine the highest and lowest marks.
 8. Calculate the percentage from the total marks.
 9. Use Math.round() to display the percentage up to two decimal places.
 10. Use String operations such as trim(), toUpperCase(), and length() to process the student's name.
 11. Assign grades using the criteria shown below.
 12. Consider the student passed only if the percentage is at least 50%.
 13. Display a performance remark based on the grade.
 14. Create a Student object in the main() method.
 15. Display the complete performance report.

Percentage	Grade
90 - 100	A+
80 - 89	A
70 - 79	B
60 - 69	C
50 - 59	D
Below 50	F (Fail)

■ Real-Time Application

College student performance analysis.

■ UML Class Diagram

```

+-----+
|          Student          |
+-----+
| - rollNumber: int         |
| - name: String            |
| - marks: int[]            |
+-----+
| + Student(roll, name, marks[]) |
| + calculateTotal(): int       |
| + calculateAverage(): double  |
+-----+
  
```

```

| + findHighest(): int           |
| + findLowest(): int           |
| + calculatePercentage(): double |
| + getGrade(): String          |
| + getRemark(): String         |
| + displayDetails(): void      |
+-----+

```

*Relationship: This experiment uses a single, self-contained class (Student) rather than a class hierarchy – the OOP focus here is on **encapsulation** (marks array and roll/name bundled with behavior) rather than inheritance or polymorphism.*

■ Source Code

```

import java.util.Arrays;

class Student {
    private int rollNumber;
    private String name;
    private int[] marks; // marks in 5 subjects

    // Constructor to initialize student details
    Student(int rollNumber, String name, int[] marks) {
        this.rollNumber = rollNumber;
        this.name = name.trim(); // remove extra spaces
        this.marks = marks;
    }

    // Calculate total marks
    public int calculateTotal() {
        int total = 0;
        for (int mark : marks) {
            total += mark;
        }
        return total;
    }

    // Calculate average marks
    public double calculateAverage() {
        return (double) calculateTotal() / marks.length;
    }

    // Find highest marks using Math.max()
    public int findHighest() {
        int highest = marks[0];
        for (int mark : marks) {
            highest = Math.max(highest, mark);
        }
        return highest;
    }

    // Find lowest marks using Math.min()
    public int findLowest() {
        int lowest = marks[0];
        for (int mark : marks) {
            lowest = Math.min(lowest, mark);
        }
        return lowest;
    }

    // Calculate percentage (assuming each subject is out of 100)
    public double calculatePercentage() {
        double percentage = (calculateTotal() / (double) (marks.length * 100)) * 100;
        return Math.round(percentage * 100.0) / 100.0; // round to 2 decimal places
    }

    // Determine grade based on percentage
    public String getGrade() {
        double percentage = calculatePercentage();
        if (percentage >= 90) return "A+";
        else if (percentage >= 80) return "A";
        else if (percentage >= 70) return "B";
        else if (percentage >= 60) return "C";
        else if (percentage >= 50) return "D";
        else return "F";
    }
}

```

```

    }

    // Determine pass/fail
    public boolean isPassed() {
        return calculatePercentage() >= 50;
    }

    // Performance remark based on grade
    public String getRemark() {
        String grade = getGrade();
        switch (grade) {
            case "A+": return "Outstanding Performance!";
            case "A":  return "Excellent Performance!";
            case "B":  return "Very Good Performance!";
            case "C":  return "Good, but can improve.";
            case "D":  return "Average, needs more effort.";
            default:   return "Needs serious improvement.";
        }
    }

    // Display complete student performance report
    public void displayDetails() {
        System.out.println("==== STUDENT PERFORMANCE REPORT =====");
        System.out.println("Roll Number   : " + rollNumber);
        System.out.println("Student Name  : " + name.toUpperCase());
        System.out.println("Name Length   : " + name.length() + " characters");
        System.out.println("Marks         : " + Arrays.toString(marks));
        System.out.println("Total Marks   : " + calculateTotal());
        System.out.printf("Average Marks : %.2f%n", calculateAverage());
        System.out.println("Highest Marks : " + findHighest());
        System.out.println("Lowest Marks  : " + findLowest());
        System.out.printf("Percentage   : %.2f%%\n", calculatePercentage());
        System.out.println("Grade        : " + getGrade());
        System.out.println("Result       : " + (isPassed() ? "PASS" : "FAIL"));
        System.out.println("Remark       : " + getRemark());
    }
}

// Main class
public class StudentPerformanceAnalyzer {
    public static void main(String[] args) {

        int[] subjectMarks = {88, 76, 95, 68, 82};

        Student student = new Student(101, " Ananya Rao ", subjectMarks);

        student.displayDetails();
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as:

```
StudentPerformanceAnalyzer.java
```

Compile the program:

```
javac StudentPerformanceAnalyzer.java
```

Run the compiled program:

```
java StudentPerformanceAnalyzer
```

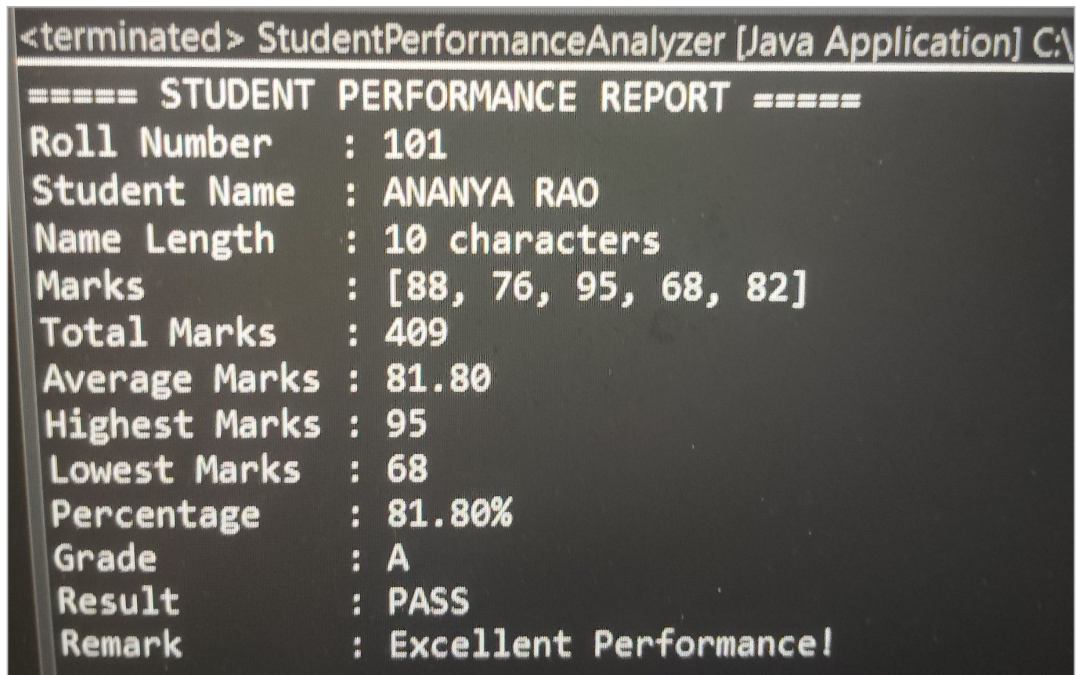
■ Output Screen

```

===== STUDENT PERFORMANCE REPORT =====
Roll Number   : 101
Student Name  : ANANYA RAO
Name Length   : 11 characters
Marks         : [88, 76, 95, 68, 82]
Total Marks   : 409
Average Marks : 81.80
Highest Marks : 95
Lowest Marks  : 68
Percentage    : 81.80%

```

```
Grade      : A
Result     : PASS
Remark     : Excellent Performance!
```



```
<terminated> StudentPerformanceAnalyzer [Java Application] C:\
===== STUDENT PERFORMANCE REPORT =====
Roll Number      : 101
Student Name     : ANANYA RAO
Name Length      : 10 characters
Marks            : [88, 76, 95, 68, 82]
Total Marks      : 409
Average Marks    : 81.80
Highest Marks    : 95
Lowest Marks     : 68
Percentage       : 81.80%
Grade           : A
Result          : PASS
Remark          : Excellent Performance!
```

Fig: Console output of StudentPerformanceAnalyzer in Eclipse IDE

■ Result

The Student Performance Analysis System was successfully implemented in Java using OOP concepts, arrays, and built-in packages. The program successfully calculates and displays the student's academic performance, including total marks, average, highest and lowest marks, percentage, grade, result, and performance remark.